Transformer 模型

第 2 章（下）

第 3 课时 本课时讲解 2.3（后半）、2.4–2.6

注意力机制 · 模型结构 · 训练与推理 · 代码实践

著者
邓杭　杨耀东

教材
《大语言模型：训练、精调与对齐》

出版
清华大学出版社



本章目录 第 3 课时

2.1 Transformer 概述
自注意力机制与 Transformer 的核心价值 第 3 页

2.2 注意力机制
序列表征、注意力概念与注意力分数 第 7 页

2.3 Transformer 模型结构
输入层、编码器、解码器与输出层 第 24 页

2.4 Transformer 训练和推理
并行训练与自回归推理 第 76 页

2.5 关键代码实践
缩放点积注意力与多头注意力的 PyTorch 实现 第 85 页

2.6 小结
本章要点与位置编码的必要性（附录 2.6.1） 第 92 页

本课时内容 2.3 Transformer 模型结构（后半）、2.4 Transformer 训练和推理、2.5 关键代码实践、2.6 小结（其余部分见第 2 课时）

2.3

Transformer 模型结构

先看翻译数据与训练目标，再逐层拆解输入层、编码器、解码器与输出层，并看清注意力层在其中的位置。

本节内容

2.3.1 机器翻译数据

2.3.2 Transformer 模型架构和训练目标

2.3.3 输入层

2.3.4 编码器

2.3.5 解码器

2.3.6 输出层

为什么用层归一化，而不是批归一化

归一化把每层的输入标准化为均值 0、方差 1 的分布，可以稳定训练，允许更大的学习率、加速收敛。

批归一化（BN）

在通道维度上归一化，统计同一位置、跨整个批量样本的均值与方差。

变长序列与 <PAD> 位置会让统计量不稳定、难以计算，因而在语言建模中效果不佳。

常用于计算机视觉的卷积网络。

层归一化（LN）

在每个词元的特征维度上归一化，每个词向量独立计算自身的均值与方差。

不受变长序列和批量大小的影响，适合语言模型。

特征通道上还有可学习的线性变换 γ、β，有助于加快收敛。

层归一化：每个词向量独立归一化

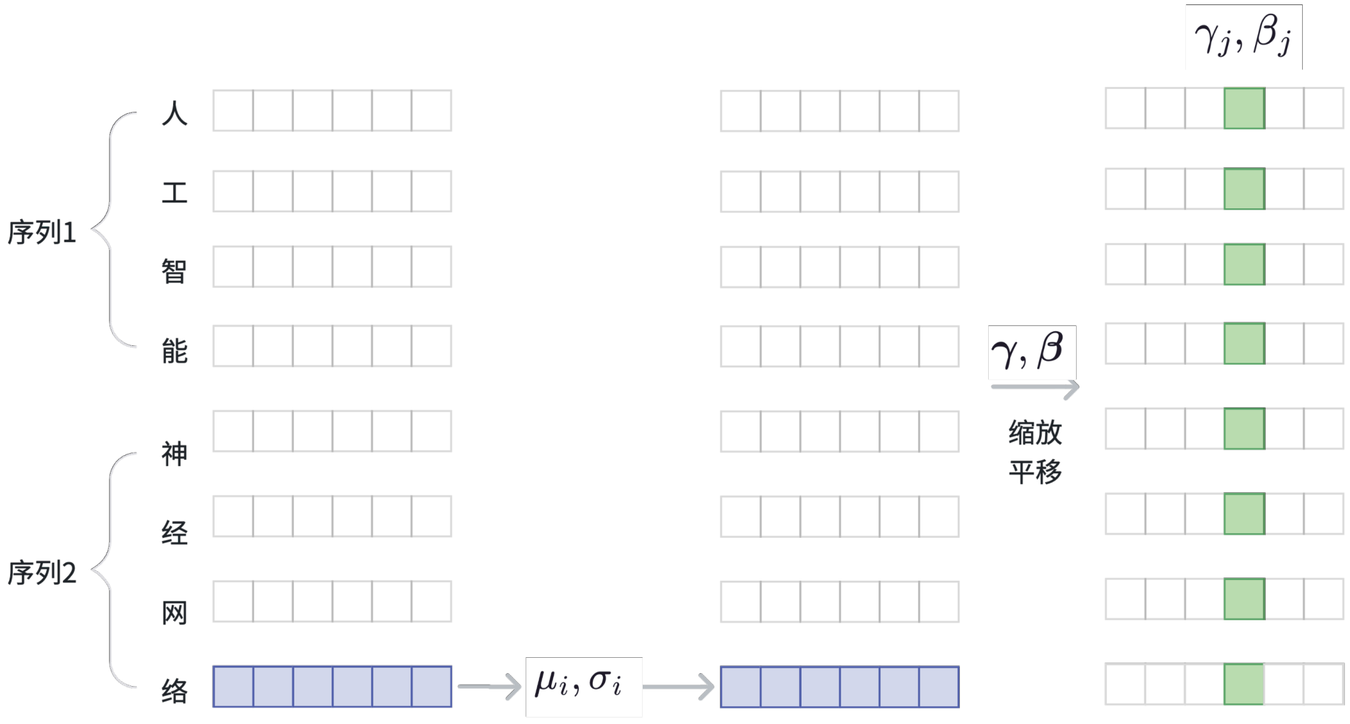


图 2.16　批量数据的层归一化

- 示例中两个序列分别为（人、工、智、能）与（神、经、网、络）。
- 每个词向量各自计算均值 μ_i 与方差 σ_i^2，不依赖其他序列。
- 归一化之后，在特征维度上做缩放 γ 与平移 β。

同一列的不同词元分属不同序列，互不参与对方的统计量——这是层归一化与批归一化的根本差别。

层归一化的计算：均值、方差与标准化

$$\mu_i = \frac{1}{d}\sum_{j=1}^{d} x_{ij} \tag{2-41}$$

$$\sigma_i^2 = \frac{1}{d}\sum_{j=1}^{d} (x_{ij} - \mu_i)^2 \tag{2-42}$$

$$\hat{\boldsymbol{x}}_i = \frac{\boldsymbol{x}_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} \tag{2-43}$$

$\boldsymbol{x}_i$ 为第 i 个词元的 d 维特征，ϵ 为小的正实数，防止分母为零导致数值溢出。

第 1 步

求均值 μ_i（2-41）

→

第 2 步

求方差 σ_i^2（2-42）

→

第 3 步

标准化 $\hat{\boldsymbol{x}}_i$（2-43）

批量数据 $\boldsymbol{X} \in \mathbb{R}^{B\times L\times d}$ 中共有 $N = B \times L$ 个词元，上述运算对每个词元分别进行。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-41)–(2-43)）

可学习的缩放与平移，以及归一化的效果

$$\tilde{x}_{ij} = \gamma_j \hat{x}_{ij} + \beta_j \tag{2-44}$$

$\boldsymbol{\gamma} \in \mathbb{R}^d$ 初始化为全 1，$\boldsymbol{\beta} \in \mathbb{R}^d$ 初始化为全 0；对第 i 个词元的第 j 维做变换。$\boldsymbol{\gamma}$ 调整尺度，$\boldsymbol{\beta}$ 调整偏移。

$$\frac{1}{d}\sum_{j=1}^{d} \hat{x}_{ij} = \frac{1}{\sqrt{\sigma_i^2+\epsilon}} \cdot \frac{1}{d}\sum_{j=1}^{d}(x_{ij}-\mu_i) = \frac{\mu_i - \mu_i}{\sqrt{\sigma_i^2+\epsilon}} = 0$$

$$\frac{1}{d}\sum_{j=1}^{d} \hat{x}_{ij}^2 = \frac{1}{\sigma_i^2+\epsilon} \cdot \frac{1}{d}\sum_{j=1}^{d}(x_{ij}-\mu_i)^2 = \frac{\sigma_i^2}{\sigma_i^2+\epsilon} \approx 1$$

由式 (2-41)、(2-42) 代入式 (2-43) 得到：归一化后的每个词向量均值为 0；当 ϵ 远小于 σ_i^2 时，方差约为 1。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-44)；均值与方差的验证由式 (2-41)–(2-43) 推出）

前馈层：升维、激活，再降维

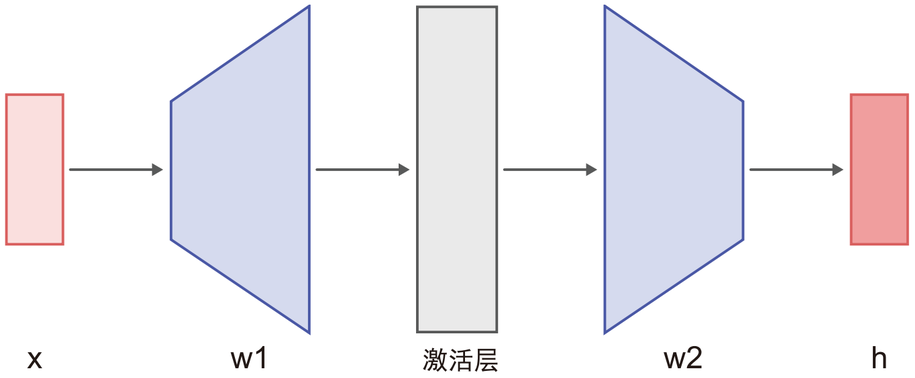


图 2.17　单个向量的前馈层计算示意图

升维

$\boldsymbol{W}_1 \in \mathbb{R}^{d\times 4d}$，$\boldsymbol{b}_1 \in \mathbb{R}^{4d}$，投影到更高维空间。

激活

$\mathrm{ReLU}(x) = \max(0, x)$，引入非线性。

降维

$\boldsymbol{W}_2 \in \mathbb{R}^{4d\times d}$，$\boldsymbol{b}_2 \in \mathbb{R}^{d}$，恢复为输入的维度。

前馈层的计算：逐位置独立进行

$$\boldsymbol{X}_{\text{up}} = \boldsymbol{X}\boldsymbol{W}_1 + \boldsymbol{b}_1 \tag{2-45}$$

$$\boldsymbol{X}_{\text{act}} = \texttt{ReLU}\left(\boldsymbol{X}_{\text{up}}\right) \tag{2-46}$$

$$\boldsymbol{X}_{\text{down}} = \boldsymbol{X}_{\text{act}}\boldsymbol{W}_2 + \boldsymbol{b}_2 \tag{2-47}$$

$$\boldsymbol{h}_i = \left(\texttt{ReLU}\left(\boldsymbol{x}_i\boldsymbol{W}_1 + \boldsymbol{b}_1\right)\right)\boldsymbol{W}_2 + \boldsymbol{b}_2 \tag{2-48}$$

式 (2-45)–(2-47) 的形状依次为 $n \times 4d$、$n \times 4d$、$n \times d$；式 (2-48) 对单个词向量 $\boldsymbol{x}_i \in \mathbb{R}^{1\times d}$ 独立计算，无需与其他词元交互。

- 升维提供更大的容量，学习更复杂的非线性特征；降维控制复杂度，并把维度恢复为 d，便于堆叠。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-45)–(2-48)）

编码器的整体：多层编码块的上下文融合

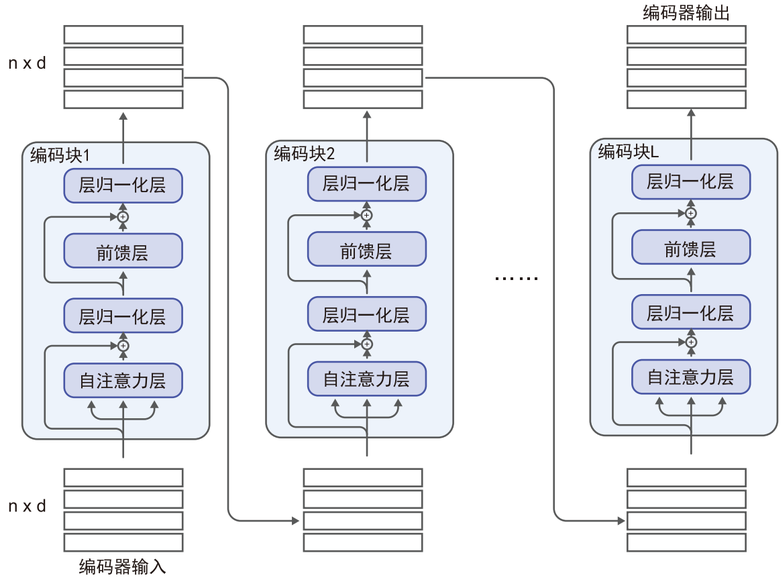


图 2.18　多层编码块计算过程

- 第一个自注意力层后，每个词元已按相关性加权融合了其他词的信息。
- 后续编码块反复做这种加权融合，捕捉更复杂的依赖关系。
- $\boldsymbol{X}_{\text{enc}} \in \mathbb{R}^{n \times d}$ 中每个词元都带有整个输入序列的语义。
- 与 RNN 编码器相比，它为每个词元保留一个编码特征，并行计算；RNN 通过递归串行计算，通常只得到一个（或一组）隐向量。

解码器：自回归生成的 L 个解码块

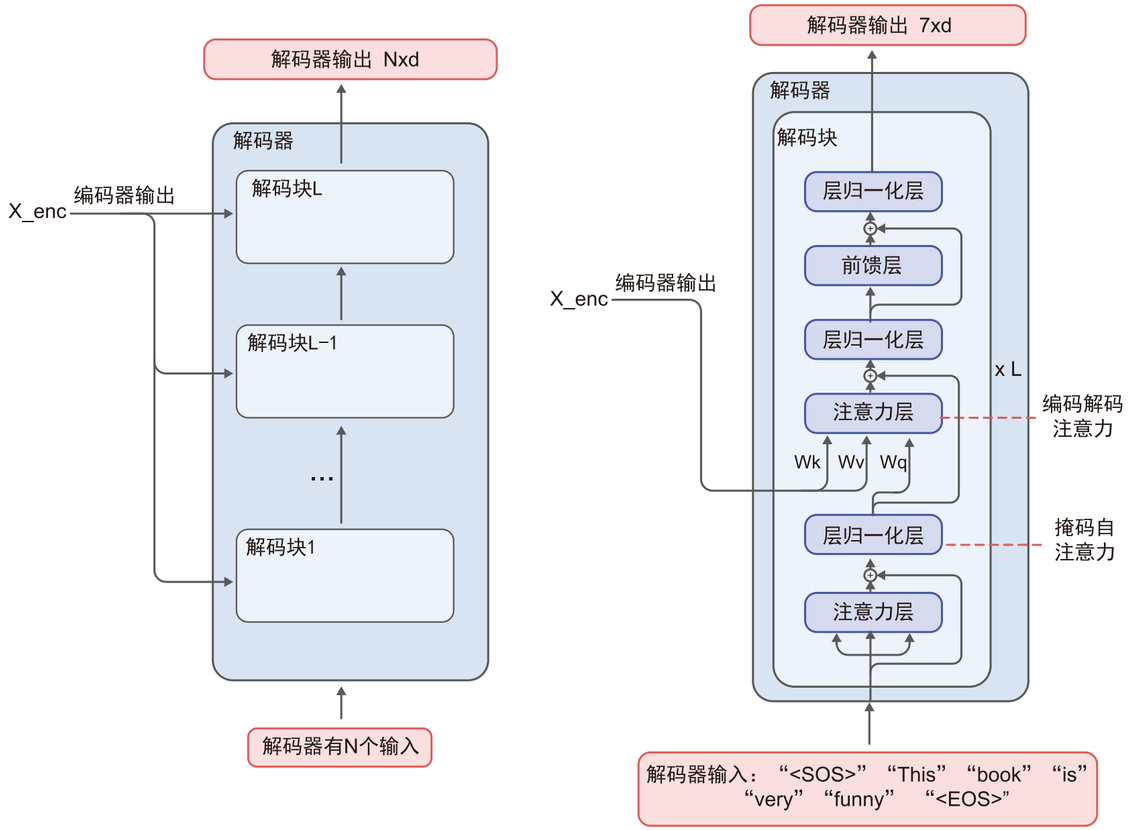


图 2.19　解码器结构，解码器的输入为目标翻译文本的词元序列

$$\boldsymbol{Y}^{(l)} = \texttt{Decoder_Block}^{(l)}\left(\boldsymbol{X}_{\text{enc}}, \boldsymbol{Y}^{(l-1)}\right) \tag{2-49}$$

第 l 个解码块同时读取编码器输出 $\boldsymbol{X}_{\text{enc}}$ 与自身输入 $\boldsymbol{Y}^{(l-1)}$；$\boldsymbol{Y}^{(0)} = \boldsymbol{Y} \in \mathbb{R}^{m\times d}$，$m$ 为目标序列长度。

- 任务：根据编码器对源语言的理解，以及目标语言前 t 个词元，预测第 $t+1$ 个词。
- 训练时，整个目标序列可以并行输入，每个位置各自预测下一个词。

解码块：三类层与残差连接

解码块在编码块的基础上多了一次“编码-解码注意力”，使解码器在生成每个词时读取源语言的信息。

子层 1

掩码自注意力层

$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ 均来自 $\boldsymbol{Y}^{(l-1)}$（同一输入）。因果掩码 $\boldsymbol{M}^{\text{dec}}$ 使位置 t 只能看到 $\leqslant t$ 的词元，防止信息泄露。

子层 2

编码-解码注意力层

$\boldsymbol{Q}$ 来自解码块特征 $\boldsymbol{Y}_{\text{dec}}$；$\boldsymbol{K}, \boldsymbol{V}$ 来自编码器输出 $\boldsymbol{X}_{\text{enc}}$，用于学习源语言与目标语言之间的对齐。

其他

LN、前馈层与残差

层归一化层（`Layer_Norm_1/2/3`）、前馈层与残差连接，与编码块相同。

依据 《大语言模型：训练、精调与对齐》第 2.3.5 节（解码块的三类层）

解码块（一）：掩码自注意力子层

$$\boldsymbol{Y}_{\text{attn}} = \texttt{Multi_Head_Attention}\left(\boldsymbol{Y}^{(l-1)}, \boldsymbol{M}^{\text{dec}}\right) \tag{2-50}$$

$$\boldsymbol{Y} = \boldsymbol{Y}_{\text{attn}} + \boldsymbol{Y}^{(l-1)} \tag{2-51}$$

$$\boldsymbol{Y}_{\text{ln1}} = \texttt{Layer_Norm_1}\left(\boldsymbol{Y}\right) \tag{2-52}$$

结构与式 (2-10)–(2-12) 的编码块注意力子层相同：先做残差相加，再归一化。区别在于掩码矩阵为因果掩码 $\boldsymbol{M}^{\text{dec}}$。

- $\boldsymbol{Y}^{(l-1)}$ 同时作为 $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ 的来源，因此这是自注意力。
- $\boldsymbol{M}^{\text{dec}}$ 决定每个位置能够看到哪些已生成的词元（见 2.3.5 掩码相关页）。

依据 《大语言模型：训练、精调与对齐》第 2.3.5 节（式 (2-50)–(2-52)）

解码块（二）：编码-解码注意力与前馈子层

$$\boldsymbol{Y}_{\text{attn}} = \texttt{Encoder_Decoder_Attention}\left(\boldsymbol{Y}_{\text{ln1}}, \boldsymbol{X}_{\text{enc}}, \boldsymbol{M}^{\text{enc-dec}}\right) \tag{2-53}$$

$$\boldsymbol{Y}' = \boldsymbol{Y}_{\text{attn}} + \boldsymbol{Y}_{\text{ln1}} \tag{2-54}$$

$$\boldsymbol{Y}_{\text{ln2}} = \texttt{Layer_Norm_2}\left(\boldsymbol{Y}'\right) \tag{2-55}$$

$$\boldsymbol{Y}_{\text{ffn}} = \texttt{Feed_Forward}\left(\boldsymbol{Y}_{\text{ln2}}\right) \tag{2-56}$$

$$\boldsymbol{Y}'' = \boldsymbol{Y}_{\text{ffn}} + \boldsymbol{Y}_{\text{ln2}} \tag{2-57}$$

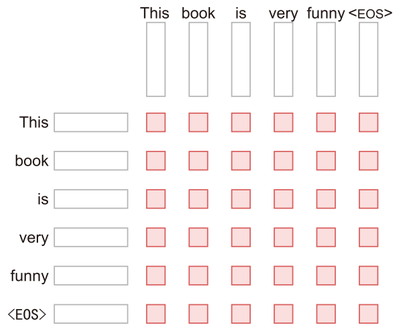
$$\boldsymbol{Y}^{(l)} = \texttt{Layer_Norm_3}\left(\boldsymbol{Y}''\right) \tag{2-58}$$

三个子层都是“先残差相加、再归一化”：$\boldsymbol{Y}'$、$\boldsymbol{Y}''$ 为残差相加后的中间结果。编码-解码注意力的 $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ 来源见下一节。

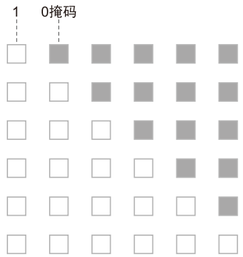
依据 《大语言模型：训练、精调与对齐》第 2.3.5 节（式 (2-53)–(2-58)）

掩码自注意力：因果掩码 M^{dec}

图 2.20　掩码自注意力层的掩码矩阵示意图



(a) 注意力分数



(b) 掩码矩阵：主对角线及以下为 1，以上为 0

$$M_{ij}^{\text{dec}} = \begin{cases} 1 & \text{if } (w_i \neq \text{<PAD>} \text{ and } w_j \neq \text{<PAD>} \\ & \quad \text{and } j \leqslant i) \\ 0 & \text{otherwise} \end{cases} \tag{2-59}$$

只允许位置 i 关注位置 1 到 i 的词元（当前及之前），屏蔽未来位置 $j > i$。

- 深灰色区域对应 $M_{ij}^{\text{dec}} = 0$，其分数随后被置为 $-\infty$（见下页）。
- 矩阵是下三角结构：位置越靠后，可读取的前文越完整。
- 同时出现的 <PAD> 条件用于批量训练时的填充位置。

图 《大语言模型：训练、精调与对齐》图 2.20；式 (2-59)

因果掩码如何作用于注意力分数

$$S_{ij} = \begin{cases} S_{ij} & \text{if } M_{ij}^{\text{dec}} = 1 \\ -\infty & \text{if } M_{ij}^{\text{dec}} = 0 \end{cases} \tag{2-60}$$

$$\boldsymbol{Y}' = \texttt{Multi_Head_Attention}\left(\boldsymbol{Y}, \boldsymbol{M}^{\text{dec}}\right) \tag{2-61}$$

$\boldsymbol{Y}'$ 为当前层的解码特征；式 (2-60) 中被屏蔽的 S_{ij} 取 $-\infty$。

$$\begin{aligned} \boldsymbol{S}_i &= (S_{i1}, \cdots, S_{ii}, -\infty, \cdots, -\infty) \\ \boldsymbol{P}_i &= \texttt{Softmax}\left(\boldsymbol{S}_i\right) = (P_{i1}, \cdots, P_{ii}, 0, \cdots, 0) \\ \boldsymbol{z}_i &= \boldsymbol{P}_i \boldsymbol{V} = \sum_{j \leqslant i} P_{ij} \boldsymbol{v}_j \end{aligned}$$

以第 i 行为例：$j > i$ 的位置权重为 0，$\boldsymbol{z}_i$ 只由位置 1 到 i 的值向量组成。

编码-解码注意力：Q 来自解码器，K、V 来自编码器

$$\boldsymbol{Q}_{\text{dec}} = \boldsymbol{Y}_{\text{dec}}\boldsymbol{W}_q + \boldsymbol{b}_q \tag{2-62}$$

$$\boldsymbol{K}_{\text{enc}} = \boldsymbol{X}_{\text{enc}}\boldsymbol{W}_k + \boldsymbol{b}_k \tag{2-63}$$

$$\boldsymbol{V}_{\text{enc}} = \boldsymbol{X}_{\text{enc}}\boldsymbol{W}_v + \boldsymbol{b}_v \tag{2-64}$$

$$\boldsymbol{S} = \frac{\boldsymbol{Q}_{\text{dec}}\boldsymbol{K}_{\text{enc}}^{T}}{\sqrt{d}} \tag{2-65}$$

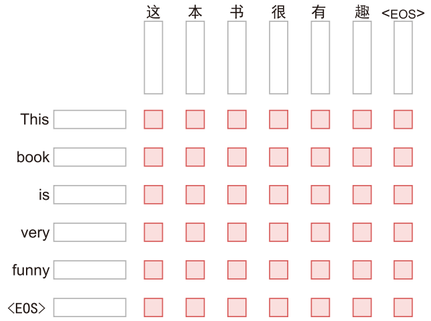
$\boldsymbol{X}_{\text{enc}} \in \mathbb{R}^{n\times d}$，$\boldsymbol{Y}_{\text{dec}} \in \mathbb{R}^{m\times d}$；$\boldsymbol{Q}_{\text{dec}} \in \mathbb{R}^{m\times d}$，$\boldsymbol{K}_{\text{enc}}, \boldsymbol{V}_{\text{enc}} \in \mathbb{R}^{n\times d}$，$\boldsymbol{S} \in \mathbb{R}^{m\times n}$。式 (2-65) 为非多头形式，实际以多头实现。

- 每个目标位置（查询）与全部源语言位置（键）计算分数，因此 $\boldsymbol{S}$ 是 $m \times n$ 的矩形矩阵。
- n 为源语言长度，m 为目标语言长度，二者不必相等。

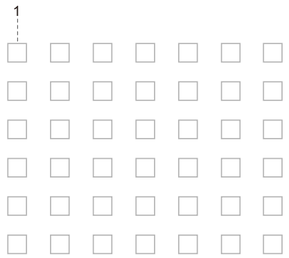
依据 《大语言模型：训练、精调与对齐》第 2.3.5 节（式 (2-62)–(2-65)）

编码-解码注意力的填充掩码

图 2.21　编码器-解码器注意力掩码矩阵（示例：源语言“这本书很有趣<EOS>”，目标语言 This book is very funny <EOS>）



(a) 注意力分数



(b) 掩码矩阵

$$M_{ij}^{\text{enc-dec}} = \begin{cases} 1 & \text{if } (w_i \neq <\text{PAD}> \\ & \quad \text{and } w_j \neq <\text{PAD}>) \\ 0 & \text{otherwise} \end{cases} \tag{2-66}$$

$M^{\text{enc-dec}} \in \{0,1\}^{m \times n}$；$w_i$ 来自目标语言第 i 个位置，w_j 来自源语言第 j 个位置。

- 与因果掩码不同：掩码里没有 $j \leqslant i$ 的条件，任何目标位置都能看到全部源语言位置。
- 矩阵为 $m \times n$，行数等于目标序列长度，列数等于源序列长度。
- 被屏蔽的列是源序列末尾的 <PAD> 填充位，避免把填充信息读入解码。

三种注意力：计算形式相同，来源与掩码不同

编码器自注意力　式 (2-10)、(2-39)

$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ 同源

三者都取自 $\boldsymbol{X}^{(l-1)}$，在源序列内部读取。

掩码 $\boldsymbol{M}^{\text{enc}}$ 屏蔽 <PAD>。

解码器掩码自注意力　式 (2-50)、(2-59)

$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ 同源

三者都取自 $\boldsymbol{Y}^{(l-1)}$，在目标序列内部读取。

掩码 $\boldsymbol{M}^{\text{dec}}$ 只看当前及之前。

编码-解码注意力　式 (2-53)、(2-66)

$\boldsymbol{Q}$ 与 $\boldsymbol{K}, \boldsymbol{V}$ 异源

$\boldsymbol{Q}$ 取自 $\boldsymbol{Y}_{\text{dec}}$，$\boldsymbol{K}, \boldsymbol{V}$ 取自 $\boldsymbol{X}_{\text{enc}}$。

掩码 $\boldsymbol{M}^{\text{enc-dec}}$ 屏蔽两端 <PAD>。

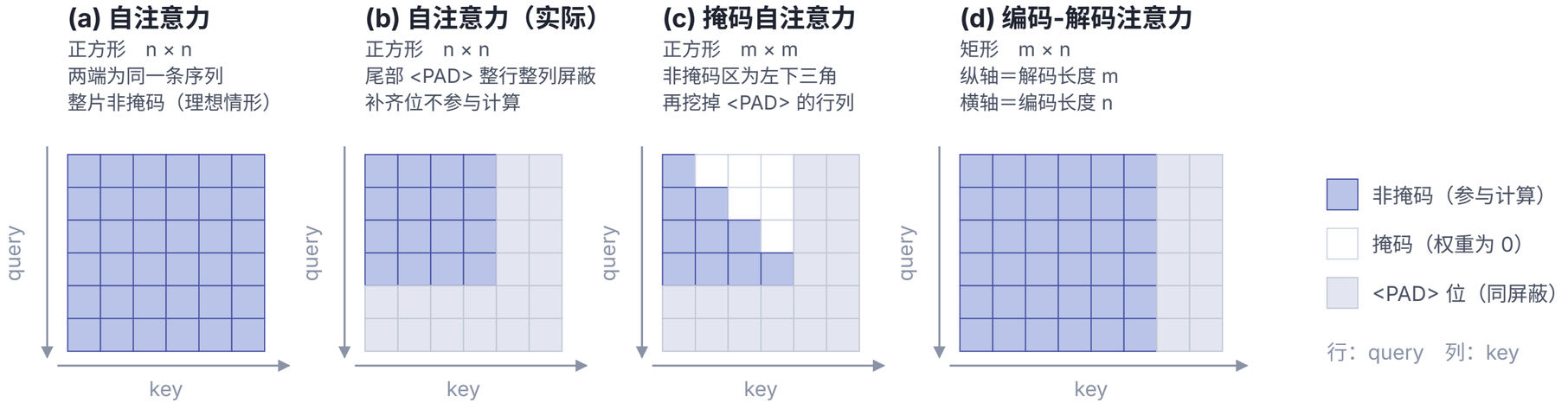
$$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \longrightarrow \boldsymbol{S} = \frac{\boldsymbol{Q}\boldsymbol{K}^{T}}{\sqrt{d}} \xrightarrow{\text{掩码}} \boldsymbol{P} = \texttt{Softmax}(\boldsymbol{S}) \longrightarrow \boldsymbol{Z} = \boldsymbol{P}\boldsymbol{V}$$

主线的四步对三者完全相同；只有掩码这一步，三张掩码矩阵的构造不同（式 (2-39)、(2-59)、(2-66)）。

注　三者的核心计算都是 $\texttt{Softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{T}}{\sqrt{d}}\right)\boldsymbol{V}$；差别只在于 $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ 从哪里来，以及掩码矩阵如何构造。

三种注意力的掩码形态

把注意力分数矩阵画成网格：**行是 query、列是 key**。矩阵的**形状**由两端长度决定，**哪一片被遮住**由掩码决定；被屏蔽的格子在 Softmax 后权重为 0。实际批次会把序列补齐到同一长度，(a) 那样的整片可见几乎不存在——(b)(c) 才是实际的样子：尾部 <PAD> 整行整列屏蔽，否则补齐位会污染注意力。



形状

自注意力两端都是序列长度，因此是方阵；(d) 两端分别是目标与源，因此是矩形，行数与列数不必相等。

方向

行永远是 query、列永远是 key。屏蔽**整行**则该 query 读不到任何东西（<PAD> 行正是如此）；屏蔽**整列**则该 key 对所有 query 都不可见。

共同点

三者都从左上角起排：因果掩码靠“只留下三角”排除未来位置，(d) 不设上下三角；<PAD> 则在右、下两侧各留一圈浅灰带。

依据第 2.3.4、2.3.5 节；掩码定义见式 (2-39)、(2-59)、(2-66)

输出层：线性投影与 Softmax

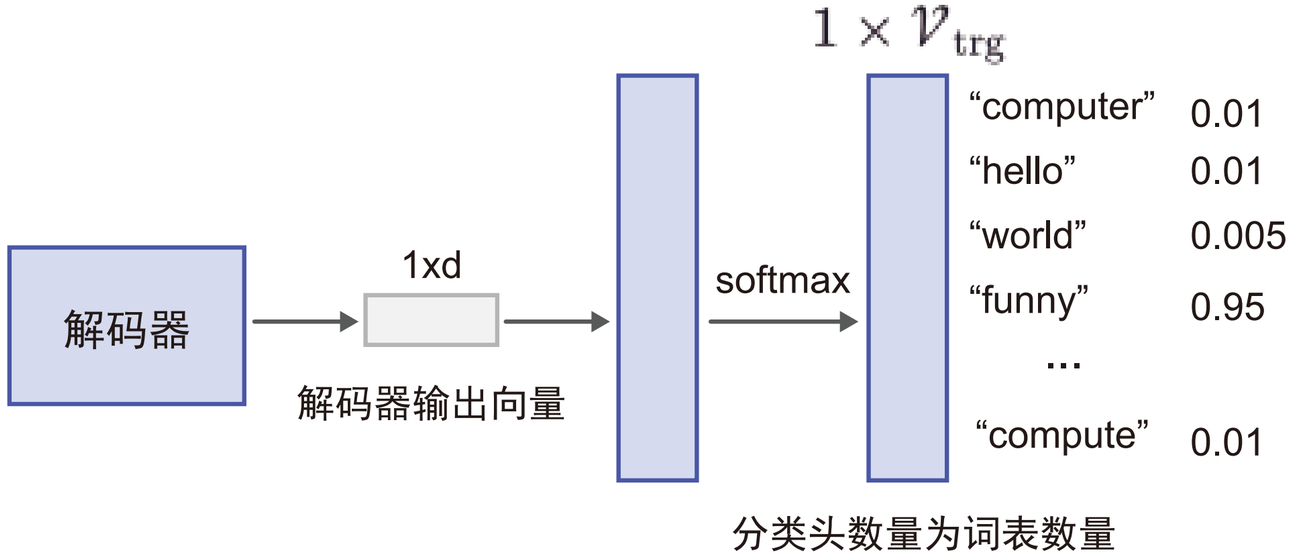


图 2.22(a)　下一个词元预测

$$p_\theta(\cdot \mid \boldsymbol{x}, \boldsymbol{y}_{\leqslant t}) = \texttt{Softmax}\left(\boldsymbol{Y}_t' \boldsymbol{W}_{\text{out}} + \boldsymbol{b}_{\text{out}}\right)$$

式 (2-67)：单个位置 t 的分布 $\in \mathbb{R}^{1\times|\mathcal{V}_{\text{trg}}|}$；整个序列为 $m \times |\mathcal{V}_{\text{trg}}|$。

- 语言分类头：$\boldsymbol{W}_{\text{out}} \in \mathbb{R}^{d\times|\mathcal{V}_{\text{trg}}|}$，$\boldsymbol{b}_{\text{out}} \in \mathbb{R}^{|\mathcal{V}_{\text{trg}}|}$。
- $|\mathcal{V}_{\text{trg}}|$ 为目标语言分词器的词表大小。
- Softmax 把投影结果转换为下一个词元的概率分布。

图 《大语言模型：训练、精调与对齐》图 2.22(a)；式 (2-67)

并行预测：一次计算所有位置的分布

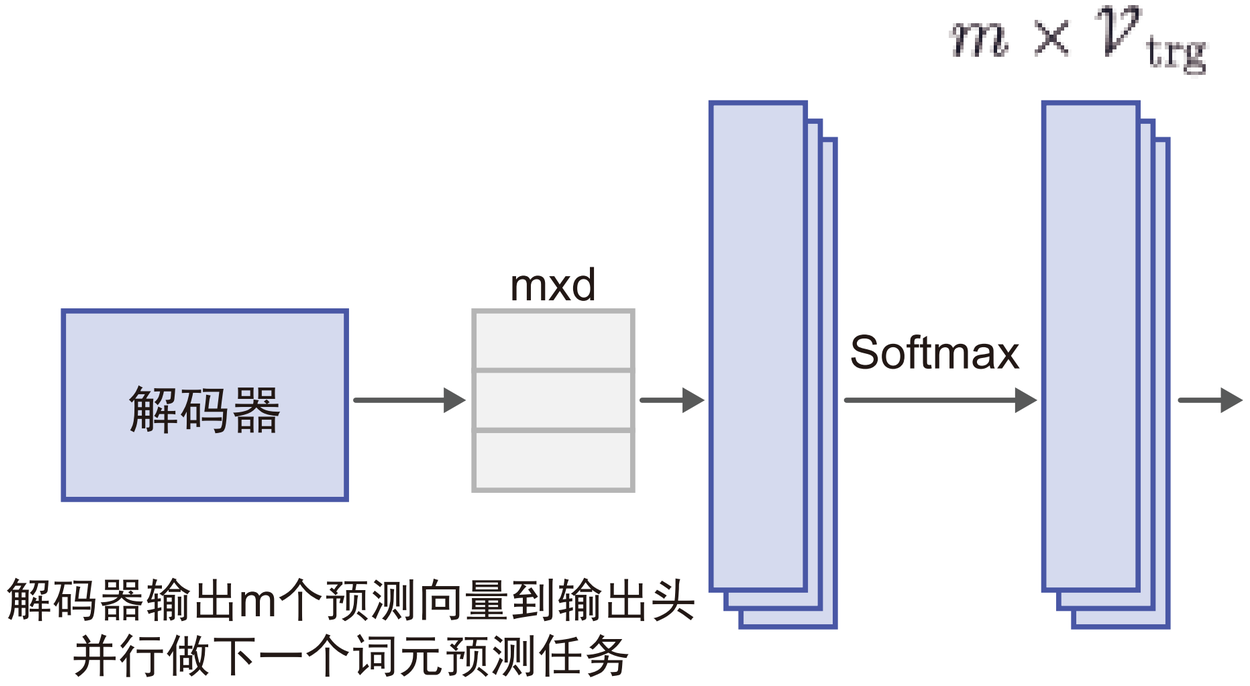


图 2.22(b)　并行预测

$$\boldsymbol{Y}' \in \mathbb{R}^{3\times d}$$

解码器输出（3 个位置）

$$\boldsymbol{Y}'\boldsymbol{W}_{\text{out}} + \boldsymbol{b}_{\text{out}} \in \mathbb{R}^{3\times|\mathcal{V}_{\text{trg}}|}$$

逐行 Softmax

每行为一个位置的概率分布

- 训练时给定完整前缀 $\boldsymbol{y}_{\leqslant m}$，式 (2-67) 对每个位置 t 并行计算 y_{t+1} 的分布。
- 可以把 Transformer 看作一个类别数极大的分类预测机器。

图 《大语言模型：训练、精调与对齐》图 2.22(b)；式 (2-67)

2.4

Transformer 训练和推理

本节内容

2.4.1 Transformer 训练

2.4.2 Transformer 推理

训练时一次前向计算并行预测所有位置；推理时则从 <SOS> 出发，逐词生成直至 <EOS>。

训练并行，推理串行

机器翻译中，模型的输入是源语言序列 $\boldsymbol{x}$ 与已生成的目标前缀 $\boldsymbol{y}_{<t}$，目标是预测下一个目标词元 y_t。训练与推理对目标序列的使用方式不同。

训练：并行

目标序列 $\boldsymbol{y}$ 整体作为解码器输入（教师强制），解码器一次前向计算所有位置的下一词元分布。

每个位置与其标签比较并计算交叉熵，反向传播后用 ADAM 优化器更新参数。

推理：串行

目标序列未知，只能从起始符 <SOS> 开始，逐词生成。

每步把预测词元追加到解码器输入，直至生成 <EOS> 或达到最大序列长度。

> **要点** 训练时的并行性来自注意力机制与掩码（2.4.1）；推理时的串行性来自“下一个词元依赖上一个词元”（2.4.2）。

一次前向计算预测所有位置

Transformer 通过单次前向计算即可同时预测目标序列中所有位置的下一词元分布，再用逐位置的交叉熵训练。

目标序列
含 <SOS>、<EOS>

→

单次前向
掩码自注意力与交叉注意力

→

逐位置分布
式 (2-67)

→

交叉熵损失
式 (2-5)

→

ADAM 更新
反向传播后更新参数

$$p_\theta(\cdot \mid \boldsymbol{x}, \boldsymbol{y}_{\leqslant t}) = \texttt{Softmax}(\boldsymbol{Y}'_t \boldsymbol{W}_{\text{out}} + \boldsymbol{b}_{\text{out}})$$

式 (2-67)：解码器第 t 个位置的输出 $\boldsymbol{Y}'_t$ 经输出层得到下一词元的分布（2.3 节）。

$$\mathcal{L}^{(k)}(\theta) = -\frac{1}{T_k} \sum_{t=1}^{T_k} \left[\log p_\theta(y_{t+1}^{(k)} \mid \boldsymbol{x}^{(k)}, \boldsymbol{y}_{\leqslant t}^{(k)}) \right]$$

式 (2-5)：样本 k 的负对数似然损失；$\frac{1}{T_k}$ 对各个位置取平均。

依据 《大语言模型：训练、精调与对齐》式 (2-5)、(2-67)

并行训练示意图

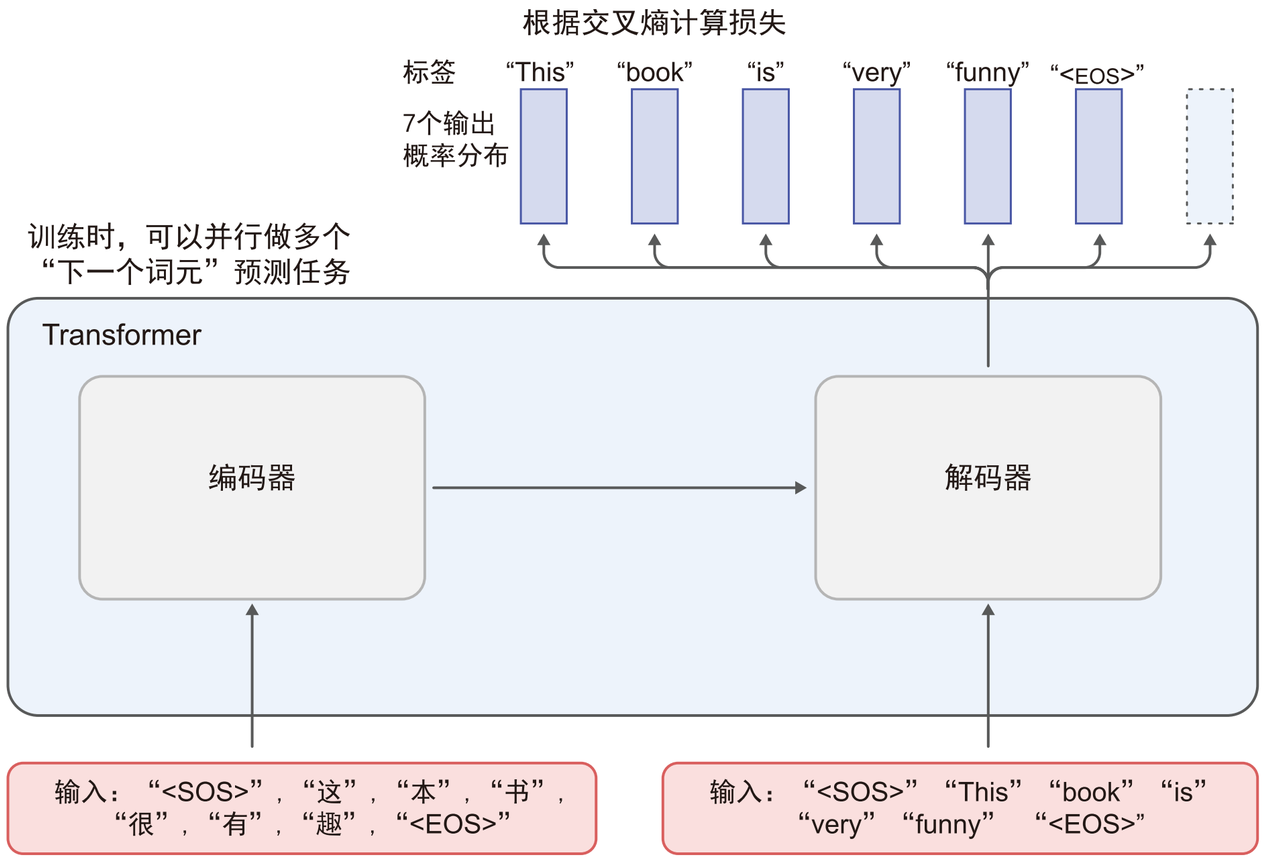


图 2.23　Transformer 并行训练示意图

- 编码器读入源语言 8 个词元（含 <SOS>、<EOS>），解码器读入目标序列 7 个词元。
- 解码器的 7 个输出位置各有一个概率分布：图中 6 个实线框标注了 This 至 <EOS>，右侧虚线框无标签。
- 末位 <EOS> 之后没有下一词元标签，不参与损失（标签为 <PAD> 的位置同样不参与），因此共 6 个预测任务。

图源 《大语言模型：训练、精调与对齐》图 2.23

输入与标签：标签左移一位

把目标序列（含 <SOS> 与 <EOS>）整体送入解码器；每个位置的标签是输入左移一位后的词元，即该位置的“下一个词元”。

位置 t	1	2	3	4	5	6	7
解码器输入	<SOS>	This	book	is	very	funny	<EOS>
标签（下一词元）	This	book	is	very	funny	<EOS>	无
参与损失	是	是	是	是	是	是	否

输入

目标序列加上 <SOS> 与 <EOS> 后共 7 个词元，整体作为解码器输入。

标签

位置 t 的标签是第 $t+1$ 个词元，即输入左移一位。

损失

末位无标签；下一词元标签为 <PAD> 的位置同样不参与计算。

依据 《大语言模型：训练、精调与对齐》第 2.4.1 节（并行训练示例）

掩码：训练时屏蔽后续词元

$$S_{ij} = \begin{cases} S_{ij} & \text{if } M^{\text{dec}}_{ij} = 1 \\ -\infty & \text{if } M^{\text{dec}}_{ij} = 0 \end{cases}$$

式 (2-60)：解码器因果掩码 M^{dec}_{ij} 在主对角线及以下为 1（保留），以上为 0（屏蔽）。

表 2.3　掩码注意力计算实例

$\boldsymbol{q}_i \boldsymbol{k}_j^T / \sqrt{d}$	$\boldsymbol{k}_1$	$\boldsymbol{k}_2$	$\boldsymbol{k}_3$
$\boldsymbol{q}_1$	s_{11}	$-\infty$	$-\infty$
$\boldsymbol{q}_2$	s_{21}	s_{22}	$-\infty$
$\boldsymbol{q}_3$	s_{31}	s_{32}	s_{33}

- $i < j$ 的分数 s_{ij} 被因果掩码置为 $-\infty$，对应 $M^{\text{dec}}_{ij} = 0$。
- Softmax 中 $e^{-\infty} = 0$，这些位置的注意力权重为 0。
- 查询 $\boldsymbol{q}_i$ 因此看不到 $\boldsymbol{y}_{>i}$，训练仍能并行计算，又不会泄露未来词元。

依据 《大语言模型：训练、精调与对齐》第 2.4.1 节；掩码定义见第 2.3.5 节

推理：从 <SOS> 开始逐词生成

训练完成后用于翻译时，目标序列未知。模型以自回归方式串行生成：每一步基于源语言 $\boldsymbol{x}$ 与已生成的前缀预测下一个词元，追加后再继续。

编码器

源语言序列只需单次前向计算，得到编码特征；之后每一步解码都复用这份特征。

解码器

初始输入只有起始符 <SOS>；输入为空则无法进行前向计算。

$$y_{\text{“This”}} = \arg\max_{w \in \mathcal{V}} p_\theta(w \mid \boldsymbol{x}, <\text{SOS}>)$$

式 (2-68)：以条件概率 $p_\theta(\cdot \mid \boldsymbol{x}, <\text{SOS}>)$ 最大的词元作为首词元（arg max，贪心选择）。

预测结果 $y_{\text{“This”}}$ 被追加到解码器输入，形成新输入 (<SOS>, This)，然后预测下一个词元。

串行推理示意图

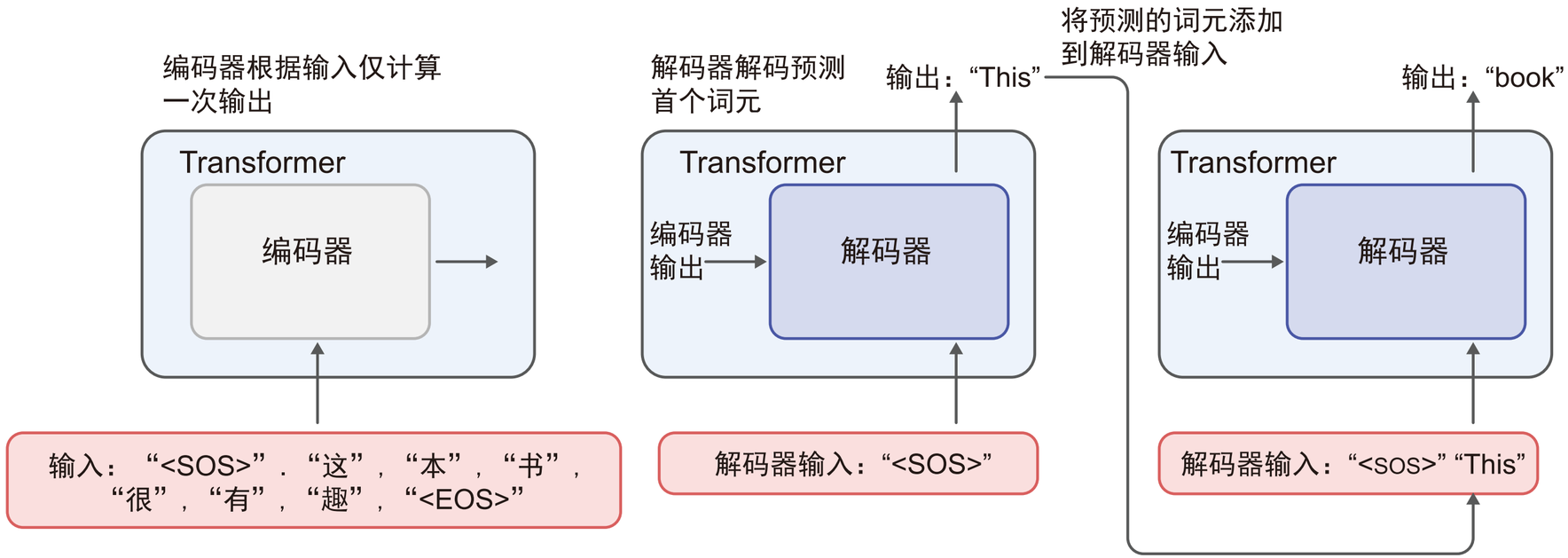


图 2.24　Transformer 串行推理

第 1 步

编码器对源语言只计算一次，输出全程不变。

第 2 步

解码器输入 <SOS>，预测首词元 This。

第 3 步

追加 This 得到 (<SOS>, This)，再预测 book。

图源 《大语言模型：训练、精调与对齐》图 2.24

逐词生成直到 <EOS>

输入 <SOS>
解码器首次输入

→

输出 This
选择概率最高的词元

→

输入 <SOS> This
追加预测词元

→

输出 book
继续预测下一词元

→

..... <EOS>
生成终止符即停止

- 基于更新后的输入序列预测下一词元，并将新词元追加至解码器输入序列。
- 迭代直至生成终止符 <EOS>，或达到最大序列长度。

代价 每次预测都要重新计算已生成词元的中间表示，计算冗余明显；4.3 节的 KV-Cache 通过缓存历史状态优化这一问题。

2.5

关键代码实践

用 PyTorch 逐段实现缩放点积注意力与多头注意力，并对照公式解释每一行代码。

本节内容

2.5.1 缩放点积注意力

2.5.2 多头注意力机制

代码 1　函数结构：一个函数完成一次注意力计算

代码 1 只有一个函数 `ScaledDotProductAttention`，加一段测试数据。计算部分共 8 行，依次是取形状、转置、点积、掩码、Softmax、加权求和、返回。

取形状
第 7 行
B, H, L, D

→

转置 K
第 8 行
`transpose(2,3)`

→

点积缩放
第 9 行
`Q @ Kt / √D`

→

掩码填充
第 10–11 行
`masked_fill`

→

Softmax
第 12 行
沿最后一维

→

加权求和
第 13–14 行
`P @ V`

函数签名

`(Q, K, V, mask)`：四个参数都是张量，`mask` 允许为 `None`。

张量形状

`(B, H, L, D)`：批、头、序列长度、每头维度。函数按张量运算，不关心具体的 B、H。

代码 1　函数与实现细节

```
 1  import torch
 2  import torch.nn as nn
 3  import math
 4  torch.manual_seed(42)
 5
 6  def ScaledDotProductAttention(Q, K, V, mask):
 7      B, H, L, D = K.size()
 8      Kt = K.transpose(2, 3)
 9      S = Q @ Kt / math.sqrt(D)
10      if mask is not None:
11          S = S.masked_fill(mask, -torch.tensor(float('inf')))
12      P = nn.functional.softmax(S, dim=-1)
13      Z = P @ V
14      return Z
```

- `K.size()` 解包出批、头、序列长度、每头维度四个量，D 只用于第 9 行的缩放。
- `transpose(2, 3)` 把键矩阵的最后两维对调，使 `Q @ Kt` 对每个批、每个头做一次矩阵乘法。
- `masked_fill` 把布尔掩码为 True 的位置写成 $-\infty$，Softmax 后这些位置的权重即为 0。
- `dim=-1` 指定沿最后一维归一化，即每一行分数各自成为一组权重。
- 函数内不做输出投影，返回的 `Z` 是加权求和的结果；投影放在多头注意力的类里。

依据 《大语言模型：训练、精调与对齐》代码 1（2.5.1 节）

代码 1　测试数据与掩码

```
15
16  # 模拟数据
17  bs = 4  # 批大小
18  heads = 8  # 注意力头数
19  seq_len = 16  # 序列长度
20  dim = 64  # 词嵌入维度
21  Q = torch.randn(bs, heads, seq_len, dim)
22  K = torch.randn(bs, heads, seq_len, dim)
23  V = torch.randn(bs, heads, seq_len, dim)
24  M = torch.zeros(1, 1, seq_len, seq_len, dtype=torch.bool)
25  out = ScaledDotProductAttention(Q, K, V, M)
```

- Q、K、V 形状为 (4, 8, 16, 64)：4 个样本、8 个头、长度 16、每头维度 64。
- 掩码形状 (1, 1, 16, 16)，通过广播作用于所有样本与所有头。
- 布尔掩码中 True 表示屏蔽，对应式 (2-40) 中 $M = 0$ 的位置；全为 False 时不屏蔽任何位置。
- 若掩码全为 True，每行分数都是 $-\infty$，Softmax 输出为 NaN。

依据 《大语言模型：训练、精调与对齐》代码 1（2.5.1 节）；掩码定义见式 (2-40)

多头注意力：切分、并行、拼接、投影

步骤 1 · 切分　式 (2-33)–(2-35)

$$\boldsymbol{Q} \to [\boldsymbol{Q}^{(1)}, \cdots, \boldsymbol{Q}^{(h)}], \quad \boldsymbol{Q}^{(i)} \in \mathbb{R}^{n \times d'}$$

$\boldsymbol{K}$、$\boldsymbol{V}$ 同理；$d' = d/h$，要求 d 能被 h 整除。

步骤 2 · 并行计算　式 (2-36)

$$\boldsymbol{Z}^{(i)} = \texttt{Softmax}\left(\frac{\boldsymbol{Q}^{(i)}\boldsymbol{K}^{(i)^T}}{\sqrt{d'}}\right)\boldsymbol{V}^{(i)}$$

每个头独立做缩放点积注意力，各头可同时计算。

步骤 3 · 拼接　式 (2-37)

$$\boldsymbol{Z} \leftarrow [\boldsymbol{Z}^{(1)}, \cdots, \boldsymbol{Z}^{(h)}] \in \mathbb{R}^{n \times d}$$

沿特征维拼接各头输出，恢复为 $n \times d$。

步骤 4 · 投影　式 (2-38)

$$\boldsymbol{O} = \boldsymbol{Z}\boldsymbol{W}_o + \boldsymbol{b}_o$$

经可学习的 $\boldsymbol{W}_o$ 与偏置 $\boldsymbol{b}_o$ 融合各头的输出。

依据 《大语言模型：训练、精调与对齐》式 (2-33)–(2-38)；代码 2（2.5.2 节）

代码 2　类的构造与张量形状变换

```
1  class MultiHeadAttention(nn.Module):
2      def __init__(self, dim, heads):
3          super(MultiHeadAttention, self).__init__()
4          self.heads = heads
5          self.dim = dim
6          self.WQ = nn.Linear(dim, dim)
7          self.WK = nn.Linear(dim, dim)
8          self.WV = nn.Linear(dim, dim)
9          self.WO = nn.Linear(dim, dim)
10
11     def forward(self, X, mask):
12         B, L, D = X.size()
13         head_dim = self.dim // self.heads  # 单头词向量维度
14         Q, K, V = self.WQ(X), self.WK(X), self.WV(X)
15         Q = Q.reshape(B, L, self.heads, head_dim).transpose(1, 2)
16         K = K.reshape(B, L, self.heads, head_dim).transpose(1, 2)
17         V = V.reshape(B, L, self.heads, head_dim).transpose(1, 2)
```

- `__init__` 里建立四个线性层，输入输出维度都是 `dim`：前三层做投影，最后一层融合各头。
- 第 13 行用整除算出每头维度；`heads` 不能整除 `dim` 时会发生截断，所以实现上要求能整除。
- 第 15–17 行的 `reshape` 把特征维拆成「头数 × 每头维度」，`transpose(1, 2)` 再把头维提到序列长度之前。
- 变换后张量为 `(B, heads, L, head_dim)`，每个头在最后两维上独立做矩阵运算。

依据 《大语言模型：训练、精调与对齐》代码 2（2.5.2 节）

代码 2　注意力计算、拼接与测试

```
18      # 缩放点积注意力计算
19      Z = ScaledDotProductAttention(Q, K, V, mask)
20      Z = Z.transpose(1, 2).reshape(B, L, self.dim)
21      O = self.WO(Z)
22      return O
23
24  bs = 4
25  heads = 8
26  seq_len = 6
27  dim = 512
28  MHA = MultiHeadAttention(dim, heads)
29  dummy_X = torch.randn(bs, seq_len, dim)
30  dummy_mask = torch.zeros(1, 1, seq_len, seq_len, dtype=torch.bool)
31  out = MHA(dummy_X, dummy_mask)
```

- 输入 X 形状为 (4, 6, 512)；切分后 Q、K、V 形状为 (4, 8, 6, 64)，即 8 个头、每头 64 维。
- 掩码形状 (1, 1, 6, 6)，全为 False，不屏蔽任何位置。
- 输出 O 的形状为 (4, 6, 512)，与输入一致，可直接接入后续层。

依据 《大语言模型：训练、精调与对齐》代码 2（2.5.2 节）

2.6 小结

回顾本章要点，并在附录 2.6.1 中完整推导位置编码对注意力的必要性。

本节内容

本章要点

2.6.1　附录：位置编码的必要性

本章要点

模型结构

注意力是核心组件

编码器自注意力、解码器掩码自注意力与编码器-解码器注意力的核心计算相同，区别主要在掩码的构造。

训练

训练并行

教师强制加掩码，使单次前向即可预测所有位置，显著提高训练效率。

推理

推理串行

需要逐词生成直至 <EOS>，每步重算已生成词元，存在冗余。

复杂度与位置

二次增长与位置编码

自注意力的时间与空间复杂度随长度二次增长；位置编码使注意力感知词序（附录）。

延伸 GPT 与 Llama 是两种代表性的纯解码器架构，分别见第 3、4 章；高效训练与推理见第 9、10 章。

不加位置编码时的注意力分数

$$S_{ij} = \boldsymbol{Q}_i \boldsymbol{K}_j^T \tag{2-69}$$

注意力分数是第 i 个查询与第 j 个键的点积，$\boldsymbol{Q}_i$、$\boldsymbol{K}_j$ 分别取 $\boldsymbol{Q}$、$\boldsymbol{K}$ 的第 i、j 行。

$$S_{ij} = (\boldsymbol{x}_i \boldsymbol{W}_q + \boldsymbol{b}_q)(\boldsymbol{x}_j \boldsymbol{W}_k + \boldsymbol{b}_k)^T \tag{2-70}$$

代入逐行投影 $\boldsymbol{Q}_i = \boldsymbol{x}_i \boldsymbol{W}_q + \boldsymbol{b}_q$、$\boldsymbol{K}_j = \boldsymbol{x}_j \boldsymbol{W}_k + \boldsymbol{b}_k$（式 (2-18)–(2-20) 的一行）。

$$S_{ij} = \boldsymbol{x}_i \boldsymbol{W}_q \boldsymbol{W}_k^T \boldsymbol{x}_j^T + \cdots \tag{2-71}$$

按分配律展开：$(\boldsymbol{x}_i \boldsymbol{W}_q)(\boldsymbol{x}_j \boldsymbol{W}_k)^T = \boldsymbol{x}_i \boldsymbol{W}_q \boldsymbol{W}_k^T \boldsymbol{x}_j^T$ 为第一项；其余含偏置的三项以 $\cdots$ 表示。

$$S_{ij} = \boldsymbol{x}_i \boldsymbol{W}_q \boldsymbol{W}_k^T \boldsymbol{x}_j^T \tag{2-72}$$

为简化推导，暂时忽略偏置项 $\boldsymbol{b}_q$、$\boldsymbol{b}_k$，只保留核心项。

依据 《大语言模型：训练、精调与对齐》第 2.6.1 节（附录）

置换不变性：词序被打乱时输出不变

$$\boldsymbol{O}_i = \sum_{j=1}^{n_k} S_{ij} \boldsymbol{V}_j$$

式 (2-73) 输出 $\boldsymbol{O}_i$ 是各值向量 $\boldsymbol{V}_j$ 以 S_{ij} 为权重的加权和（此处忽略缩放与 Softmax）。

$$\boldsymbol{O}_i = \sum_{j=1}^{n_k} S_{ij} \boldsymbol{V}_j = \sum_{j'=1}^{n_k} S_{ij'} \boldsymbol{V}_{j'}$$

式 (2-74) 若同一词元 $\boldsymbol{x}_j = \boldsymbol{x}_{j'}$ 换到位置 j'：S_{ij} 只依赖 $\boldsymbol{x}_i$、$\boldsymbol{x}_j$，$\boldsymbol{V}_j$ 只依赖 $\boldsymbol{x}_j$，每一项都不变；求和只改变顺序，所以 $\boldsymbol{O}_i$ 保持不变。

结论 只有注意力、没有位置信息时，打乱词序不改变输出，模型无法区分“大模型”与“模大型”。

依据 《大语言模型：训练、精调与对齐》第 2.6.1 节（式 (2-73)、(2-74)）

打乱词序：无位置编码不变，有位置编码改变

同一组三个词元，只把“大”与“模”的位置对调。左：分数只由词向量本身决定，输出逐项相同；右：位置向量 $\boldsymbol{p}_i$ 已随词元进入分数，对调后 $\boldsymbol{S}$ 不同，输出随之改变。

不加位置编码

分数只由词向量本身决定，与位置无关

序列 ①

大	模	型

序列 ② 交换“大”与“模”

模	大	型

输出 ①

0.55	0.75	0.62

输出 ②

0.55	0.75	0.62

打乱词序，输出逐项相同

加入位置编码

位置向量随词元一起进入分数，换位后不再相同

序列 ①

大 p_1	模 p_2	型 p_3

序列 ② 交换“大”与“模”

模 p_1	大 p_2	型 p_3

输出 ①

0.55	0.75	0.62

输出 ②

0.68	0.84	0.48

打乱词序，输出随之改变

加入位置编码后的注意力分数

$$S_{ij} = \boldsymbol{Q}_i \boldsymbol{K}_j^T \tag{2-75}$$

输入换成加入位置编码后的 $\boldsymbol{x}_i + \boldsymbol{p}_i$（见 2.3.3 节输入层），$\boldsymbol{Q}_i$、$\boldsymbol{K}_j$ 的定义不变。

$$S_{ij} \approx (\boldsymbol{x}_i + \boldsymbol{p}_i)\boldsymbol{W}_q \boldsymbol{W}_k^T (\boldsymbol{x}_j + \boldsymbol{p}_j)^T \tag{2-77}$$

忽略偏置项，与式 (2-72) 的简化相同。

$$S_{ij} = ((\boldsymbol{x}_i + \boldsymbol{p}_i)\boldsymbol{W}_q + \boldsymbol{b}_q)((\boldsymbol{x}_j + \boldsymbol{p}_j)\boldsymbol{W}_k + \boldsymbol{b}_k)^T \tag{2-76}$$

把 $\boldsymbol{x}_i$ 替换为 $\boldsymbol{x}_i + \boldsymbol{p}_i$，把 $\boldsymbol{x}_j$ 替换为 $\boldsymbol{x}_j + \boldsymbol{p}_j$。

位置项进入注意力分数

$$S_{ij} = \boldsymbol{x}_i \boldsymbol{W}_q \boldsymbol{W}_k^T \boldsymbol{x}_j^T + \boldsymbol{x}_i \boldsymbol{W}_q \boldsymbol{W}_k^T \boldsymbol{p}_j^T + \boldsymbol{p}_i \boldsymbol{W}_q \boldsymbol{W}_k^T \boldsymbol{x}_j^T + \boldsymbol{p}_i \boldsymbol{W}_q \boldsymbol{W}_k^T \boldsymbol{p}_j^T$$

式 (2-78) 展开为四项：词与词、词对位置、位置对词、位置与位置。其中 $\boldsymbol{p}_i \boldsymbol{W}_q \boldsymbol{W}_k^T \boldsymbol{p}_j^T$ 同时涉及两个位置。

$$\boldsymbol{O}_i = \sum_{j=1}^{n_k} S_{ij} \boldsymbol{V}_j \neq \sum_{j=1}^{n_k} S_{ij'} \boldsymbol{V}_{j'}$$

式 (2-79) 当 $\boldsymbol{p}_j \neq \boldsymbol{p}_{j'}$ 时，$\boldsymbol{K}_j \neq \boldsymbol{K}_{j'}$，$S_{ij} \neq S_{ij'}$，输出随之改变。例如 (大, 模, 型) 与 (模, 大, 型) 中的“型”，加入位置编码后输出不同。

依据 《大语言模型：训练、精调与对齐》第 2.6.1 节（式 (2-78)、(2-79)）

位置编码点积的推导（一）：定义与展开

设位置 i、j 的二维绝对位置编码 $\boldsymbol{p}_i, \boldsymbol{p}_j \in \mathbb{R}^2$，$\theta_k$ 表示完整位置编码的第 k 个角度。

$$\boldsymbol{p}_i = (\sin(i\theta_k),\ \cos(i\theta_k))$$
$$\boldsymbol{p}_j = (\sin(j\theta_k),\ \cos(j\theta_k))$$

第 1 步　写出两个编码的分量：第一分量为正弦，第二分量为余弦。

$$\begin{aligned}\boldsymbol{p}_i\boldsymbol{p}_j^T &= \sin(i\theta_k)\sin(j\theta_k) \\ &\quad + \cos(i\theta_k)\cos(j\theta_k)\end{aligned} \tag{2-80}$$

第 2 步　按点积定义对应分量相乘再相加：第一分量得 $\sin(i\theta_k)\sin(j\theta_k)$，第二分量得 $\cos(i\theta_k)\cos(j\theta_k)$。

这一结果含有两个角度 $i\theta_k$、$j\theta_k$，还不能直接看出它只依赖 $j-i$，需要用三角公式化简（下页）。

位置编码点积的推导（二）：化为 cos((j−i)θ)

$$\cos(A-B)=\cos A\cos B+\sin A\sin B$$

第 3 步　两角差的余弦公式（三角恒等式）。

$$\begin{aligned}\boldsymbol{p}_i\boldsymbol{p}_j^{T}&=\sin(i\theta_k)\sin(j\theta_k)\\&\quad+\cos(i\theta_k)\cos(j\theta_k)\\&=\cos\big((j-i)\theta_k\big)\end{aligned}\tag{2-81}$$

第 5 步　由第 4 步的结果，位置编码的点积只取决于差值 $j-i$。

$$\begin{aligned}\cos\big((j-i)\theta_k\big)&=\cos(j\theta_k-i\theta_k)\\&=\cos(j\theta_k)\cos(i\theta_k)\\&\quad+\sin(j\theta_k)\sin(i\theta_k)\end{aligned}$$

第 4 步　令 $A=j\theta_k$、$B=i\theta_k$，则 $(j-i)\theta_k=A-B$。右端两项与式 (2-80) 的两项相同，只是乘法顺序不同。

依据　《大语言模型：训练、精调与对齐》第 2.6.1 节（式 (2-80)、(2-81)）

只依赖相对位置 $j-i$

- 式 (2-81) 的结果只取决于差值 $j-i$，与 i、j 各自的绝对位置无关。
- 对完整位置编码，每个角度 θ_k 的分量对都给出 $\cos((j-i)\theta_k)$，求和后仍只依赖 $j-i$。
- 位置编码把“相对距离”直接写进注意力分数，打破了置换不变性。
- 正文末段指出：在纯解码器结构中，位置编码对长度泛化未必绝对关键（NoPE）。

示例：取 $\theta_k = 0.5$

位置对 (i, j)	$j-i$	$\cos((j-i)\theta_k)$
(1, 2)	1	0.878
(3, 4)	1	0.878
(1, 3)	2	0.540
(2, 4)	2	0.540

> **注** 式 (2-81) 针对不加投影矩阵的点积。注意力中还有 $\boldsymbol{W}_q\boldsymbol{W}_k^T$，此时分数不再严格只依赖 $j-i$，但位置信息已显式进入分数。

文献 Kazemnejad et al., *The Impact of Positional Encoding on Length Generalization in Transformers*, arXiv 2023（NoPE）

思考题与实践题

1 自注意力的时间与空间复杂度各是多少？当序列长度从 n 翻倍到 $2n$ 时，两者分别变成原来的几倍？

2 表 2.3 中 s_{ij}（$i < j$）为何置为 $-\infty$？这对 $\boldsymbol{q}_i$ 有什么影响？

3 如何把 Transformer 去掉编码器，直接用于双语翻译任务的学习？解码器的输入与注意力层要相应做哪些改动？

4 生成 m 个词元需要几次解码器前向计算？与训练相比，慢在哪里？

5 掩码自注意力里，查询 $\boldsymbol{q}_i$ 究竟在检索什么？为什么它的检索范围必须截断到位置 i？

6 位置编码点积只依赖 $j - i$：相对距离越大，注意力分数会呈现什么特点？这与“远距离依赖更难学”是什么关系？

7 实践：用第 2.5 节的代码实现一个双语翻译 Transformer，跑通“编码器 + 解码器 + 输出层”的完整前向与训练。

8 实践：推导层归一化的梯度（对 $\boldsymbol{x}_i$、$\boldsymbol{\gamma}$、$\boldsymbol{\beta}$），并实现一个可反向传播的 LayerNorm 类。